

Lexorion Project Study Guide

Lexorion Project Study Guide

This document explains Lexorion from first principles. Use it to relearn the project, refresh the machine learning concepts, and prepare for interviews.

1. Project In One Sentence

Lexorion is a contract risk intelligence system that reads legal contract text, detects risky clauses, groups them into business-friendly risk categories, and shows a contract-level risk profile in a dashboard.

2. What Problem Are We Solving?

Contracts are long and difficult to review manually. Important risks may be hidden inside legal language.

Lexorion tries to help a user answer:

- Does this contract contain risky clauses?
- What type of risk is present?
- Where is the risky clause in the text?
- How confident is the system?
- What should a human reviewer inspect first?

Lexorion is not legal advice. It is an AI-assisted review tool.

3. What Is CUAD?

CUAD stands for Contract Understanding Atticus Dataset.

It is a public dataset of legal contracts where important clauses are labeled. Think of it as:

```
contracts + expert-highlighted clauses + clause labels
```

Example:

```
"Either party may terminate this agreement with 30 days notice."
```

CUAD may label this as:

Lexorion uses CUAD as the training and evaluation foundation.

4. Why 41 Labels Become 8 Risk Categories

CUAD has 41 detailed legal clause labels. Some are too legal-specific for a business dashboard.

Lexorion maps selected CUAD labels into 8 broader business-facing risk categories.

Example:

Limitation of Liability

Cap on Liability

Uncapped Liability

Liquidated Damages

-> Liability Risk

This is called a taxonomy mapping.

Important limitation:

The mapping is manually designed. That means it is useful and transparent, but subjective. In production, legal experts should validate it.

5. The 8 Risk Categories

Risk Category | Meaning

Liability Risk | Who pays, and how much, if something goes wrong

IP Risk | Intellectual property ownership, licenses, and usage rights

Termination Risk | How the contract can end and what happens after

Indemnification | Who compensates whom for losses, claims, or damages

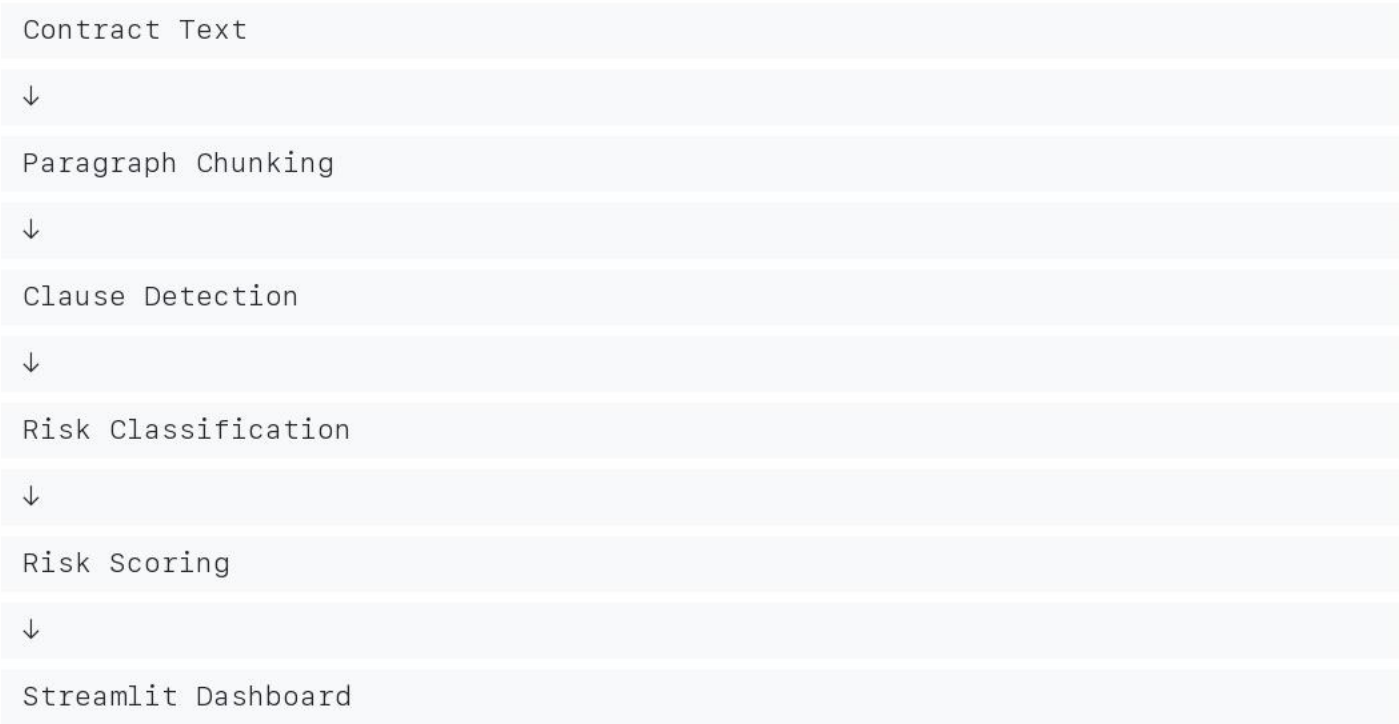
Exclusivity & Restrictions | Clauses limiting business activity or competition

Change of Control | What happens if ownership/control changes

Revenue & Financial Risk | Minimum commitments, revenue sharing, pricing, audit rights

Renewal & Expiration | Contract duration, auto-renewal, expiration, notice periods

6. Project Architecture



Longer version:



7. Important Files

``configs/category_mapping.yaml``

Defines the 8 business risk categories and maps CUAD labels into them.

This is the business logic of the project.

``src/data_pipeline/download_cuad.py``

Downloads the CUAD dataset and saves it locally.

``src/data_pipeline/parse_cuad.py``

Converts CUAD from question-answer format into a clean table.

``src/data_pipeline/chunk_contracts.py``

Splits full contracts into paragraphs and labels each paragraph based on whether a CUAD answer span appears inside it.

This creates the model-ready dataset.

``src/models/baseline_detector.py``

Trains a fast baseline model using TF-IDF and logistic regression.

This gives us a benchmark before using expensive or complex models.

``src/models/clause_detector.py``

Designed for DeBERTa transformer training. This is the stronger local ML model, but it is heavier to train.

``src/models/llm_classifier.py``

Uses an LLM such as Claude or OpenAI to classify, extract, and summarize contract risk clauses.

``src/models/hybrid_pipeline.py``

Combines local ML and LLM reasoning.

The idea:

Use local model for fast/cheap screening.

Use LLM for difficult or uncertain cases.

`src/evaluation/metrics.py`

Computes model metrics such as precision, recall, F1, AUPR, and Jaccard similarity.

`src/evaluation/model\_comparison.py`

Compares different approaches:

- Baseline
- DeBERTa
- LLM
- Hybrid

`src/dashboard/app.py`

Streamlit dashboard. This is what users and recruiters can see.

8. ML Refresher

Supervised Learning

The model learns from examples where the correct answer is known.

In Lexorion:

```
Input: contract paragraph
```

```
Label: does this paragraph contain liability risk? yes/no
```

Binary Classification

A model predicts one of two classes:

```
0 = clause not present
```

```
1 = clause present
```

Lexorion trains one binary classifier per risk category.

Example:

```
liability_risk classifier
```

```
ip_risk classifier
```

```
termination_risk classifier
```

Multi-Label Classification

A paragraph can belong to multiple categories at the same time.

Example:

```
One paragraph may contain both:
```

```
- revenue_risk
```

```
- termination_risk
```

Lexorion currently handles this by training separate binary classifiers for each category.

Train / Validation / Test Split

Data is split into three parts:

Split | Purpose

Train | Used to train the model

Validation | Used to tune choices like thresholds

Test | Used once for final evaluation

Important rule:

Do not tune your model on the test set. That is considered cheating because the test set should represent unseen data.

Feature Extraction

Machine learning models need numbers, not raw text.

TF-IDF converts text into numeric features.

Example:

```
"terminate agreement"
```

```
-> numerical vector
```

TF-IDF

TF-IDF means Term Frequency-Inverse Document Frequency.

It gives higher importance to words that are meaningful in one document but not common everywhere.

In Lexorion, the baseline model uses TF-IDF to turn paragraphs into features.

Logistic Regression

Despite the name, logistic regression is often used for classification.

It predicts a probability:

```
P(liability_risk present) = 0.82
```

Then we choose a threshold:

```
if probability >= threshold:
```

```
    predict risk present
```

```
else:
```

```
    predict risk absent
```

Decision Threshold

The default threshold is usually 0.5, but that is not always best.

In Lexorion, we improved the baseline by choosing category-specific thresholds using the validation split.

This improved macro F1 from:

```
0.539 -> 0.638
```

Precision

Precision answers:

```
When the model says "risk present", how often is it correct?
```

High precision means fewer false alarms.

Formula:

```
precision = true positives / (true positives + false positives)
```

Recall

Recall answers:

```
Of all real risky clauses, how many did the model catch?
```

High recall means fewer missed risks.

Formula:

```
recall = true positives / (true positives + false negatives)
```

F1 Score

F1 balances precision and recall.

It is useful when classes are imbalanced.

Formula:

```
F1 = 2 * (precision * recall) / (precision + recall)
```

False Positive

The model says risk exists, but the label says it does not.

In legal review, false positives waste reviewer time.

False Negative

The model says no risk, but a real risk exists.

In legal review, false negatives are dangerous because the system missed something important.

9. Current Dataset Stats

Processed data:

```
408 contracts
```

```
34,820 paragraph chunks
```

```
3,922 risky paragraphs
```

```
11.3% positive/risky paragraph rate
```

Train/validation/test data exists in:

```
data/processed/paragraphs_train.parquet
```

```
data/processed/paragraphs_val.parquet
```

```
data/processed/paragraphs_test.parquet
```

10. Current Baseline Model

Model:

TF-IDF + Logistic Regression

Why we use it:

- Fast
- Free
- Easy to explain
- Gives a benchmark before DeBERTa or LLMs

Current macro results:

Macro F1: 0.638
Macro Precision: 0.628
Macro Recall: 0.682
Latency: <1s
Cost: \$0 local

11. Category-Level Baseline Results

Category	F1	Precision	Recall
Change of Control	0.783	0.890	0.699
Indemnification	0.753	0.659	0.879
Liability Risk	0.706	0.652	0.769
IP Risk	0.705	0.804	0.628
Renewal & Expiration	0.689	0.808	0.600
Revenue Risk	0.512	0.428	0.638
Exclusivity	0.494	0.396	0.656
Termination Risk	0.466	0.388	0.584

Main finding:

The baseline is decent for a simple model, but weaker on termination, exclusivity, and revenue risk.

12. Why Precision Was Initially Low

At threshold 0.5, the model was too eager. It flagged too many paragraphs as risky.

After threshold tuning on validation data:

```
Macro F1 improved from 0.539 to 0.638
```

Lesson:

Model training is not just choosing an algorithm. The decision threshold matters.

13. What Is DeBERTa?

DeBERTa is a transformer language model.

Compared with TF-IDF, DeBERTa understands more context.

Example:

TF-IDF mostly sees important words:

```
terminate, liability, exclusive
```

DeBERTa can better understand sentence meaning and legal context.

Why we want it:

- Better context understanding
- Potentially higher F1
- Better performance on subtle clauses

Tradeoff:

- Slower to train
 - Needs more compute
 - Better done on GPU or Colab
-

14. What Is An LLM Classifier?

An LLM classifier uses a model like Claude/OpenAI to read a paragraph and return structured JSON.

Example:

```
{  
  "classification": "PRESENT",  
  "risk_level": "high",  
  "summary": "The vendor has broad liability exposure.",  
  "confidence": 0.87  
}
```

Why use it:

- Gives explanations
- Extracts clause text
- Handles difficult language

Tradeoff:

- Costs money
- Slower
- Needs API keys
- Needs careful evaluation

15. What Is The Hybrid Pipeline?

The hybrid idea:

```
Use local model first.  
If confident, accept prediction.  
If uncertain or category is hard, send to LLM.
```

Why this is strong:

- Lower cost than LLM-only
- More explainable than local-only
- More practical for real products

Interview phrase:

> I designed the system as a cost-aware hybrid pipeline: a local classifier handles fast screening, while the LLM is reserved for uncertain or high-value cases that require extraction and explanation.

16. Unknown Risk Discovery

The current 8 categories are closed taxonomy categories.

But some risks may not fit.

Example:

```
Data Privacy Risk
```

Lexorion includes an example unknown-risk candidate:

```
examples/unknown_risk_candidates.json
```

The idea:

```
If a paragraph seems risky but does not fit the 8 categories,  
flag it as an unknown risk candidate for human review.
```

Important:

The system should not automatically create new categories without human review. That would become messy and hard to trust.

17. What The Dashboard Shows

The dashboard has three tabs:

Risk Analysis

Shows:

- Demo contract text
- Risk scores
- Flagged clauses
- Unknown risk candidates

Model Comparison

Shows:

- Overall model metrics
- Per-category F1 comparison

- Best approach by category

Dataset Explorer

Shows:

- Number of paragraphs
 - Number of contracts
 - Risk category distribution
 - Train/validation/test split
 - Paragraph length distribution
-

18. Current Project Strengths

- Real legal dataset
 - Clear business problem
 - Clean data pipeline
 - Passing tests
 - Working dashboard demo
 - Real baseline model metrics
 - Good hybrid architecture plan
 - Honest README status
-

19. Current Limitations

- DeBERTa not trained yet
 - LLM evaluation not complete
 - Hybrid comparison not complete
 - PDF parsing not production-ready
 - Manual taxonomy mapping needs legal validation
 - Demo output is currently precomputed
 - Deployment not complete
-

20. Next Technical Steps

Recommended order:

1. Add baseline error analysis report.
 2. Train DeBERTa for one category first.
-

3. Compare DeBERTa against baseline.
 4. Run small LLM evaluation sample.
 5. Add hybrid routing logic with real predictions.
 6. Deploy Streamlit demo.
 7. Add screenshots and final README polish.
-

21. Interview Explanation

Short version:

> Lexorion is a contract risk intelligence system built on CUAD. We map detailed legal clause labels into 8 business-facing risk categories, create paragraph-level training data, and evaluate models that detect risky clauses. The current version includes a Streamlit demo and a TF-IDF logistic regression baseline with validation-tuned thresholds, achieving 0.638 macro F1. The next step is comparing DeBERTa and LLM-based classification against that baseline.

Longer version:

> The project starts with CUAD, a legal contract dataset in QA format. I parse it into structured records, map selected CUAD labels into broader risk categories, chunk contracts into paragraphs, and create multi-label paragraph-level targets. I then train one binary classifier per risk category. The baseline uses TF-IDF and logistic regression, with thresholds tuned on the validation split. The dashboard shows risk scores, flagged clauses, and unknown-risk candidates. The architecture is designed to evolve into a hybrid system where a local model handles fast screening and an LLM handles uncertain cases requiring explanation.

22. Questions You Should Be Ready To Answer

Why map 41 labels to 8 categories?

Because CUAD labels are legally detailed, while business users need broader, easier-to-read risk groups. We preserve traceability to original CUAD labels but simplify the dashboard output.

Is manual mapping a limitation?

Yes. It is subjective and should be validated by legal experts. But it is transparent and useful for a portfolio/product prototype.

Why use a baseline model?

A baseline tells us what simple methods can achieve. We should only use DeBERTa or LLMs if they beat the baseline or provide useful explanations.

Why tune thresholds?

The default 0.5 threshold was not optimal. Tuning thresholds on validation data improved macro F1 from 0.539 to 0.638.

Why not use only an LLM?

LLMs are slower and cost money. A hybrid system can use a local model for cheap screening and an LLM for hard cases.

What is the biggest weakness right now?

The model comparison is incomplete. We have a strong baseline, but still need DeBERTa, LLM, and hybrid evaluation.

23. Commands To Remember

Run tests:

```
venv/bin/python -m pytest tests -q
```

Train baseline:

```
venv/bin/python -m src.models.baseline_detector
```

Generate model comparison:

```
venv/bin/python -m src.evaluation.model_comparison
```

Run dashboard:

```
venv/bin/streamlit run src/dashboard/app.py
```

24. Glossary

Term | Meaning

CUAD | Contract Understanding Atticus Dataset

Clause | A specific section or provision in a contract

Label | The correct category assigned to a text example

Taxonomy | A structured category system

Baseline | A simple model used as a benchmark

TF-IDF | Text feature method based on word importance

Logistic Regression | Classification model that predicts probabilities

Threshold | Probability cutoff for predicting class 1

Precision | How often positive predictions are correct

Recall | How many true positives are caught

F1 | Balance of precision and recall

False Positive | Model flags risk but label says no risk

False Negative | Model misses a true risk

DeBERTa | Transformer language model for text understanding

LLM | Large language model such as Claude or GPT

Hybrid Pipeline | System combining local ML and LLM reasoning

Streamlit | Python framework for building data apps